authorization failures should be handled.

For example, in chapter 23 the cookie authentication scheme was used implicitly by ASP.NET Core Identity. The cookie authentication handler in this case authenticates users by looking for a cookie and redirects users to the login or “access denied” pages for authentication or authorization failures.

In listing 25.1 you registered the JWT Bearer authentication scheme. The JWT bearer authentication handler reads tokens from the `Authorization` header and returns `401` and `403` responses for authentication or authorization failures.

When you register only a single authentication scheme, such as in listing 25.1, ASP.NET Core automatically sets that as the default, but it’s possible to register multiple authentication schemes. This is particularly common if you are using OpenID Connect with a traditional web app, for example. In these cases you can choose which scheme is used for authentication events or authentication failures and how the schemes should interact.

Using multiple authentication schemes can be confusing, so it’s important to follow the documentation closely when configuring authentication for your app. You can read more about authentication schemes at http://mng.bz/5w1a. If you need only a single scheme, you shouldn’t have any problems, but otherwise, here be dragons!